

DAY 2: CODING EXERCISE

Build a Bitcoin Explorer in Python!

- ★ Show blockchain info
- ★ Check wallet balance
- ★ List recent transactions
- ★ Decode a transaction
- ★ Display block details

Files: `bootcamp-code/day2/explorer_starter.py`

HOW APPS TALK TO BITCOIN

Your App (Python) ↔ Bitcoin Core (bitcoind)
JSON-RPC
Port 18443

JSON-RPC = Send JSON commands, get JSON responses

Same commands as `bitcoin-cli` but programmatic!

JSON-RPC REQUEST

```
{  
  "jsonrpc": "1.0",  
  "id": "myapp",  
  "method": "getblockchaininfo",  
  "params": []  
}
```

Response:

```
{  
  "result": {"chain": "regtest", "blocks": 101},  
  "error": null  
}
```

THE RPC HELPER

```
import requests, json

def rpc(method, params=None, wallet=None):
    url = "http://127.0.0.1:18443/"
    if wallet:
        url = f"{url}wallet/{wallet}"

    data = json.dumps({
        "jsonrpc": "1.0", "id": "explorer",
        "method": method, "params": params or []
    })

    resp = requests.post(url, data=data,
                        auth=("bootcamp", "bootcamp123"))
```

USING THE HELPER

```
# Get blockchain info
info = rpc("getblockchaininfo")
print(f"Chain: {info['chain']}")
print(f"Blocks: {info['blocks']}")

# Get wallet balance
balance = rpc("getbalance", [], wallet="alice")
print(f"Alice has {balance} BTC")
```

EXERCISE SETUP

```
# Make sure Bitcoin Core is running  
bitcoin-cli -regtest getblockchaininfo
```

```
# Go to exercise folder  
cd bootcamp-code/day2
```

```
# Install requests  
pip install requests
```

```
# Run the starter  
python explorer_starter.py
```

CHALLENGE 1: BLOCKCHAIN INFO

```
def show_blockchain_info():  
    """  
    TODO:  
    1. Call rpc("getblockchaininfo")  
    2. Print: chain, blocks, difficulty  
    """  
  
    # YOUR CODE HERE  
    pass
```

Hint: Result has keys `'chain'`, `'blocks'`, `'difficulty'` Bitcoin Day 2

CHALLENGE 1: SOLUTION

```
def show_blockchain_info():  
    info = rpc("getblockchaininfo")  
  
    print("=== Blockchain Info ===")  
    print(f"Chain:          {info['chain']}")  
    print(f"Blocks:           {info['blocks']}")  
    print(f"Difficulty: {info['difficulty']}")
```


CHALLENGE 2: WALLET BALANCE

```
def show_wallet_balance(wallet_name):  
    """  
    TODO:  
    1. Load wallet (try/except)  
    2. Call rpc("getbalance", [], wallet=wallet_name)  
    3. Print the balance  
    """  
    pass
```

CHALLENGE 2: SOLUTION

```
def show_wallet_balance(wallet_name):  
    try:  
        rpc("loadwallet", [wallet_name])  
    except:  
        pass # Already loaded  
  
    balance = rpc("getbalance", [], wallet=wallet_name)  
  
    print(f"=== Wallet: {wallet_name} ===")  
    print(f"Balance: {balance} BTC")
```

CHALLENGE 3: LIST TRANSACTIONS

```
def list_transactions(wallet_name, count=5):  
    """  
    TODO:  
    1. Call rpc("listtransactions", ["*", count],  
                wallet=wallet_name)  
    2. For each tx: print direction, amount, txid  
    """  
    pass
```

Hint: `tx['category']` is 'receive', 'generate', 'immature' or 'send' Bitcoin Day 2

CHALLENGE 3: SOLUTION

```
def list_transactions(wallet_name, count=5):
    try:
        rpc("loadwallet", [wallet_name])
    except:
        pass

    txs = rpc("listtransactions", ["*", count],
              wallet=wallet_name)

    for tx in txs:
        if tx['category'] in ('receive', 'generate', 'immature'):
            direction = "IN "
        else:
            direction = "OUT"

        print(f"{direction} {tx['amount']:+.8f} BTC")
```

CHALLENGE 4: DECODE TX

```
def decode_transaction(txid):  
    """  
    TODO:  
    1. Call rpc("getrawtransaction", [txid, True])  
    2. Print inputs (vin) and outputs (vout)  
    """  
    pass
```

Hint: Check if `'coinbase' in vin` for mining
reward

CHALLENGE 4: SOLUTION

```
def decode_transaction(txid):
    tx = rpc("getrawtransaction", [txid, True])

    print(f"Size: {tx['size']} bytes")

    print("\nInputs:")
    for vin in tx['vin']:
        if 'coinbase' in vin:
            print("    COINBASE (mining reward)")
        else:
            print(f"    From: {vin['txid'][:20]}...")

    print("\nOutputs:")
    for vout in tx['vout']:
        addr = vout['scriptPubKey'].get('address', '?')
```

CHALLENGE 5: BLOCK DETAILS

```
def show_block(blockhash=None):  
    """  
    TODO:  
    1. If no hash: rpc("getbestblockhash")  
    2. Call rpc("getblock", [blockhash, 1])  
    3. Print: height, hash, time, tx count  
    """  
    pass
```

CHALLENGE 5: SOLUTION

```
def show_block(blockhash=None):  
    if blockhash is None:  
        blockhash = rpc("getbestblockhash")  
  
    block = rpc("getblock", [blockhash, 1])  
  
    print(f"=== Block #{block['height']} ===")  
    print(f"Hash: {block['hash'][:32]}...")  
    print(f"Time: {block['time']}")  
    print(f"Transactions: {block['nTx']}")
```


BONUS: 2-OF-2 MULTISIG

Why Multisig matters for Lightning:

- ★ Lightning channels use **2-of-2 multisig**
- ★ Both Alice AND Bob must sign to spend
- ★ Neither can steal funds alone
- ★ Foundation of trustless payment channels!

Exercise: `bootcamp-code/day2/03_multisig.sh`

DAY 2 SUMMARY

You learned:

- ★ JSON-RPC communication with Bitcoin Core
- ★ Python requests to query the blockchain
- ★ Building a Bitcoin explorer from scratch
- ★ Transaction structure (inputs, outputs)
- ★ Block structure (header, transactions)
- ★ Multisig basics (prep for Lightning!)

Tomorrow: Lightning Network!

HOMework

- ★ Complete all 5 explorer challenges
- ★ Try the multisig bonus exercise
- ★ Add a new feature to your explorer:
- ★ Show UTXO set for a wallet
- ★ Calculate total fees in a block
- ★ Search transactions by address

Solution: `explorer_solution.py`